



## **MEMBERS**

| <b>Name</b>           | <b>SAP ID</b>    |
|-----------------------|------------------|
| <b>Ayush Negi</b>     | <b>590015410</b> |
| <b>Ayush Kumar</b>    | <b>590014539</b> |
| <b>Aditi Vijay</b>    | <b>590018228</b> |
| <b>Mayank Tanishk</b> | <b>590015863</b> |
| <b>Mayank Parmar</b>  | <b>590017039</b> |
| <b>Sakshi Singh</b>   | <b>590012402</b> |
| <b>Samiksha Rajan</b> | <b>590016234</b> |
| <b>Satyam Pandey</b>  | <b>590016569</b> |



## INDEX

| Section                        | Content Description                                                        | Page No.      |
|--------------------------------|----------------------------------------------------------------------------|---------------|
| <b>1. Project Overview</b>     | <b>Title:</b> Placement Drive Management System                            | <b>3</b>      |
| <b>2. Problem Definition</b>   | <b>Statement:</b> Addressing coordination gaps in recruitment workflows.   | <b>3</b>      |
| <b>3. Stakeholder Analysis</b> | <b>Target Users:</b> Admin (Coordinator) and Student roles.                | <b>3 - 4</b>  |
| <b>4. Domain Entities</b>      | <b>Core Objects:</b> Company, Student, Drive, Application, Interview Slot. | <b>4 - 5</b>  |
| <b>5. Operational Workflow</b> | <b>End-to-End Flow:</b> From Drive Creation to Interview Scheduling.       | <b>6</b>      |
| <b>6. Scope Definition</b>     | <b>Included:</b> CRUD, Shortlisting, Reporting, Validations.               | <b>6</b>      |
|                                | <b>Excluded:</b> External API integration, Resume parsing.                 | <b>7</b>      |
| <b>7. Unit 06 Alignment</b>    | Justification of OOP, Layered Architecture, and JDBC use.                  | <b>7</b>      |
| <b>8. Team Organization</b>    | <b>Role Split:</b> Division of members into Set 1 through Set 4.           | <b>7 - 8</b>  |
| <b>9. Project Backlog</b>      | <b>Prioritized Tasks:</b> Phase-wise breakdown (High/Med/Low).             | <b>8 - 11</b> |

# Object Oriented Programming (CSEG1044)

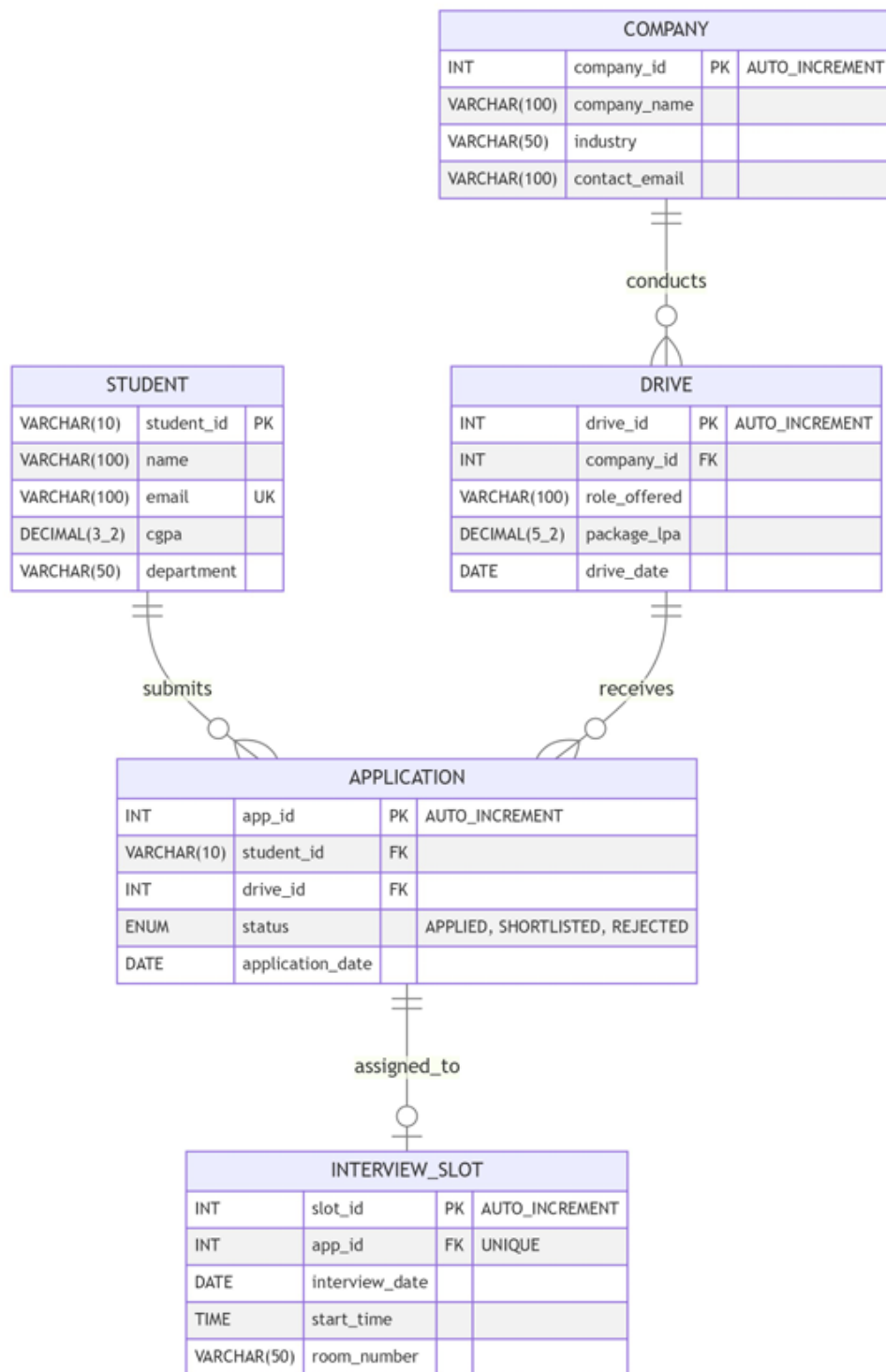
## Phase 0: Proposal and Backlog

### **Project Title: Placement Drive Management System.**

- **Problem Statement:** Placement drive information is difficult to track when company records, student applications, and schedules are managed separately. Managing placement drives involves coordinating multiple companies, hundreds of student applications, and complex interview schedules. Currently, tracking these in separate files leads to data fragmentation and scheduling conflicts. This Java desktop application centralizes these records into a single structured system to streamline the recruitment workflow from registration to final interview assignment.
- **Target Users & Their Capabilities:**
- **Admin (Placement Coordinator):** Has full access to the system. Can add/edit Company details, create new placement Drives, view all Student applications, update application statuses (e.g., Pending, Shortlisted, Rejected), and assign specific Interview Slots to shortlisted students.
- **Student:** Has restricted access. Can view their own profile, browse active placement Drives, submit an

Application for a drive, and view their application status and assigned interview slots.

- **Core Entities (The Data Model):**
- **Student:** Stores studentId, name, email, cgpa, department, graduationYear.
- **Company:** Stores companyId, name, industry, contactEmail, website.
- **Drive:** Stores driveId, companyId (foreign key), roleOffered, packageDetails, minCgpaRequired, driveDate.
- **Application:** Stores applicationId, studentId, driveId, applicationDate, status (Enum: APPLIED, SHORTLISTED, REJECTED, HIRED).
- **InterviewSlot:** Stores slotId, applicationId, interviewDate, startTime, endTime, interviewerName, roomNumber.



- **Primary End-to-End Workflow (The "Happy Path"):**
- **Registration:** The Admin logs in and registers a new "Company" in the system.
- **Drive Creation:** The Admin creates a new "Drive" associated with that Company, specifying the role, package, and date.
- **Application:** A "Student" logs into their dashboard, sees the active Drive, and clicks "Apply". The system creates an "Application" record.
- **Review & Shortlist:** The Admin views all applications for the Drive, reviews the student's profile, and changes the application status to "Shortlisted".
- **Scheduling:** The Admin selects the shortlisted Application and creates an "InterviewSlot", assigning a specific time and room. The Student can now see this schedule on their dashboard.
- **Scope Boundaries:**
- **In-Scope (Version 1):** User authentication (basic login), CRUD operations for all 5 core entities, role-based dashboards (Admin vs. Student), application status tracking, interview scheduling, and basic validations (e.g., preventing a student from applying to the same drive twice).

- **Out-of-Scope (Version 1):** Real-time email/SMS notifications, automated resume PDF parsing, complex matching algorithms, integration with external platforms (like LinkedIn or Naukri), and complex student eligibility filtering (to keep the initial SQL queries manageable).
- **2. Detailed Team Role Assignment**
- **To ensure equal distribution of work and to hit all course requirements, assign your group members to the following specific tracks:**

| Team Set                      | Core Focus              | Specific Deliverables for the Project                                                                                                                              |
|-------------------------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Set 1<br>(Systems Analysts)   | Requirements & Design   | Draft the final Use Case descriptions, create the Use Case Diagram, design the UML Class Diagram, and maintain the README.md documentation throughout the project. |
| Set 2<br>(Backend Developers) | Domain Model & Services | Code the plain Java objects (POJOs) for the 5 entities. Write the Service classes that handle business logic (e.g., ApplicationService.java which                  |

| Team Set                       | Core Focus             | Specific Deliverables for the Project                                                                                                                                                 |
|--------------------------------|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                |                        | checks if a student already applied before saving).                                                                                                                                   |
| Set 3<br>(Database Engineers)  | Database & DAO Layer   | Design the SQL table schema (DDL). Write the DAO (Data Access Object) classes using JDBC to execute INSERT, SELECT, UPDATE, and DELETE queries. Manage database connection utilities. |
| Set 4<br>(Frontend Developers) | Swing UI & Integration | Design the Java Swing GUI (Frames, Panels, Buttons, Tables). Wire the UI buttons to call the methods in the Service layer. Ensure the app is packaged correctly for the final demo.   |

- **Comprehensive Project Backlog**
- **This is the step-by-step task list your team will follow. It proves to your teacher that you have thought about how to build the app sequentially.**



- **Phase 0 & 1 Tasks (Planning & Modeling)**
- **Task 1:** Write and format the final Phase 0 proposal document.
- **Task 2:** Set up the shared Git repository and initial folder structure (model, dao, service, ui).
- **Task 3:** Create the primary Use Case Diagram (showing Admin and Student actors).
- **Task 4:** Create the UML Class Diagram showing attributes, methods, and relationships (1-to-many between Company and Drive, etc.).
- **Phase 2 & 3 Tasks (Database & Backend Implementation)**
- **Task 5:** Write the SQL script (schema.sql) to create the 5 tables with Primary and Foreign Keys.
- **Task 6:** Create the Java Entity classes (Encapsulated POJOs with getters/setters) in the model package.
- **Task 7:** Implement the DatabaseConnection utility class using JDBC.
- **Task 8:** Implement CompanyDAO and DriveDAO (CRUD operations).
- **Task 9:** Implement StudentDAO and ApplicationDAO (CRUD operations).
- **Phase 4 Tasks (Business Logic & UI Implementation)**

- **Task 10:** Implement basic Login UI and authentication logic.
- **Task 11:** Build the Admin Dashboard UI (Data tables to view Companies, Drives, and Applications).
- **Task 12:** Build the Student Dashboard UI (View available drives, Apply button).
- **Task 13:** Implement the "Shortlist" business logic and UI update.
- **Task 14:** Build the Interview Scheduling UI and backend logic (Insert into InterviewSlot table).
  
- **Phase 5 Tasks (Finalization & Testing)**
- **Task 15:** Test all workflows to ensure UI actions correctly update the database.
- **Task 16:** Clean up code, add comments, and finalize the README.md with "How to Run" instructions.
- **Task 17:** Compile the project into an executable JAR file for the final evaluation.
  
- **Academic Justification (Why this fits the Unit 06 Capstone)**
- **This project strictly adheres to the core learning outcomes of the Object-Oriented Programming course:**

- **OOP Principles:** It utilizes Encapsulation (data hiding in model classes), Abstraction (hiding database complexity behind DAO interfaces), and modularity through a layered architecture.
- **Relational Database Mapping:** It requires designing a normalized database and mapping those relational tables to Java objects, fulfilling the data persistence requirements.
- **Standard Technologies:** It comprehensively utilizes Java SE, Swing for event-driven GUI development, and JDBC for database connectivity, which are the exact technical pillars evaluated in this semester.